

RUSH

Propuesta de arquitectura para el proyecto

Organización de CSS, componentes, páginas y flujo Git/GitHub

Objetivo

Definir una estructura común, fácil de entender y segura para trabajar en equipo, sin rehacer lo que ya funciona y evitando que cada rama termine creando su propia arquitectura.

Propuesta para revisión y aprobación del equipo

1. Idea principal

La propuesta separa el proyecto en tres niveles. Esto permite conservar un CSS propio por página sin duplicar estilos globales ni componentes compartidos.

Nivel	Qué contiene	Ejemplos
GLOBAL	Reglas usadas en todo RUSH	colores.css, styles.css
COMPARTIDO	Componentes presentes en muchas páginas	navbar.css, footer.css
PÁGINA	Estilos exclusivos de una vista	contacto.css, nosotros.css, 404.css

Regla sencilla

Si se usa en todo el sitio → GLOBAL. Si se reutiliza en varias páginas → COMPARTIDO. Si pertenece únicamente a una página → CSS DE ESA PÁGINA.

2. Estructura propuesta

```
Proyecto-Cinema/  
|  
├─ README.md  
├─ index.html  
|  
├─ assets/  
|   └─ img/  
|       └─ icons/  
|  
├─ css/  
|   └─ global/  
|       └─ colores.css  
|       └─ styles.css  
|  
|   └─ componentes/  
|       └─ navbar.css  
|       └─ footer.css  
|  
|   └─ paginas/  
|       └─ home.css  
|       └─ nosotros.css  
|       └─ contacto.css  
|       └─ suscripcion.css  
|       └─ 404.css
```

```
|   ├── galerias.css
|   └── detalle.css
|
└── paginas/
    ├── 404.html
    ├── contacto.html
    ├── nosotros.html
    ├── inicio-sesion.html
    ├── suscripcion.html
    ├── peliculas/
    |   ├── detalle.html
    |   └── categorias/
    └── series/
        ├── detalle.html
        └── categorias/
```

3. Responsabilidad de cada archivo

Archivo	Responsabilidad
css/global/colores.css	Variables de identidad visual. No colocar colores hexadecimales repetidos en cada página.
css/global/styles.css	Reset, tipografías, body y reglas realmente globales.
css/componentes/navbar.css	Solo Navbar y su responsive.
css/componentes/footer.css	Solo Footer y su responsive.
css/paginas/home.css	Hero, sugeridos, categorías y contenido propio de Home.
css/paginas/nosotros.css	Equipo, avatares, tarjetas y disposición de Nosotros.
css/paginas/contacto.css	Formulario y layout exclusivo de Contacto.
css/paginas/suscripcion.css	Planes, beneficios y formulario de suscripción.
css/paginas/404.css	Número 404, barra decorativa, botón y responsive propio.
css/paginas/galerias.css	Grillas, cards y categorías de películas/series.
css/paginas/detalle.css	Detalle de películas y series.

4. Qué evitamos con esta organización

- Duplicar colores, tipografías o estilos del body en varios archivos.
- Modificar Navbar o Footer desde el CSS de una página.
- Crear una arquitectura diferente en cada rama feat/*.
- Que un cambio en Contacto rompa Nosotros o 404.
- Tener que corregir el mismo componente en muchos archivos.
- Conflictos innecesarios entre compañeros trabajando al mismo tiempo.

5. Orden recomendado de carga de CSS

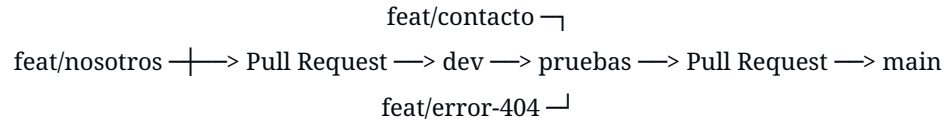
Ejemplo para paginas/contacto.html:

```
<link rel="stylesheet" href="../css/global/colores.css">
<link rel="stylesheet" href="../css/global/styles.css">
<link rel="stylesheet" href="../css/componentes/navbar.css">
<link rel="stylesheet" href="../css/componentes/footer.css">
<link rel="stylesheet" href="../css/paginas/contacto.css">
```

6. Flujo Git/GitHub propuesto

Idea de ramas

MAIN = versión revisada y aprobada. DEV = base común e integración. FEAT/* = trabajo individual o por funcionalidad.



Reglas de trabajo

- Toda nueva rama feat/* debe nacer desde una dev actualizada.
- Las ramas feat/* deben regresar a dev mediante Pull Request.
- Main recibe únicamente cambios revisados y estables.
- La arquitectura general no se modifica dentro de una página aislada.
- Si falta algo global en una feat/*, primero se verifica si dev está atrasada.
- Antes de git add, siempre ejecutar git status y agregar solo los archivos relacionados con el cambio.

7. Plan para corregir la arquitectura actual

1. Congelar temporalmente cambios estructurales dentro de feat/contacto y otras ramas de página.
2. Revisar y aprobar un único PR de arquitectura hacia dev.
3. Corregir en dev assets/img y assets/icons como carpetas reales.
4. Incorporar colores.css y styles.css como capa global.
5. Ubicar Navbar y Footer aprobados como componentes compartidos.
6. Conservar CSS propios para páginas como contacto.css, nosotros.css y 404.css.
7. Actualizar las ramas feat/* desde la nueva dev.
8. Continuar cada página tocando principalmente su HTML y su CSS específico.

8. Qué NO propone este plan

- No borrar ni rehacer el trabajo aprobado.
- No obligar a meter todos los estilos en un único CSS.
- No impedir que cada página tenga su propio CSS.
- No cambiar la forma de usar Bootstrap, Flexbox o nesting.
- No mezclar directamente main dentro de una feat/* para corregir estructura.

9. Beneficios para RUSH

Beneficio	Impacto
Más claridad	Cada archivo tiene una responsabilidad fácil de explicar.
Menos conflictos	Cada integrante modifica principalmente su página y su CSS.
Identidad consistente	Los colores y reglas globales se administran en un solo lugar.
Componentes reutilizables	Navbar y Footer mantienen el mismo comportamiento en todo el sitio.
Mejor mantenimiento	Un cambio global se hace una vez, no en cada página.
Mejor presentación	La estructura del repositorio es fácil de entender para el equipo y para la evaluación.

10. Decisiones que debería aprobar el equipo

- ☐ Separar CSS en global / componentes / páginas.
- ☐ Mantener un CSS propio por página cuando tenga estilos específicos.
- ☐ Mantener Navbar y Footer como componentes compartidos.
- ☐ Usar dev como base común de las ramas de trabajo.
- ☐ Usar Pull Requests para integrar feat/* en dev.
- ☐ Usar main únicamente para versiones revisadas y estables.

Propuesta de aprobación

Si el equipo aprueba estas decisiones, se realiza un único ajuste estructural en dev. Después cada integrante actualiza su rama y continúa trabajando sin reconstruir carpetas ni duplicar estilos.

RUSH - Propuesta de organización del proyecto